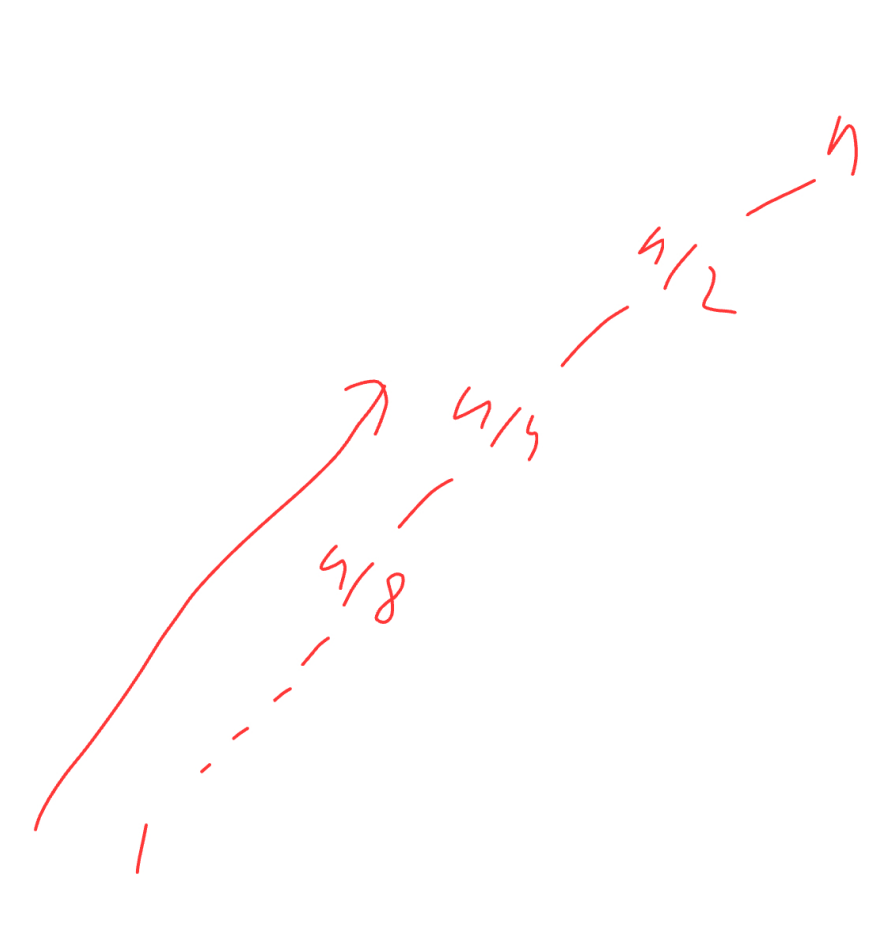


$\underbrace{1, 2, 4, 8, 16 - - - - - ⑦}_{H}$

$ar^{2,3,4}$

$ar^{H-1} = n$

$1 \cdot 2^{H-1} = n$

$H-1 = \log_2 n$

$\sqrt{n}$

$n = 10^6$

$\sqrt{n} = 10^3$ $\log n = 6$

Array.

0	1	2	3	4	5
1	3	2	5	10	9

→ Contiguous Memory Allocation.

0	1	2	3	4	5	
x	x+4	x+8	x+12	x+16	x+20	
1	3	②	5	10	9	X

vector <int> arr;

$x + 5*4$

Random Access : O(1)

→ Insert at the end : → O(1)

Insert at the beginning: O(n)

Insert at the middle : O(n)

Del from the end : O(1)

Del from the beginning : O(n)

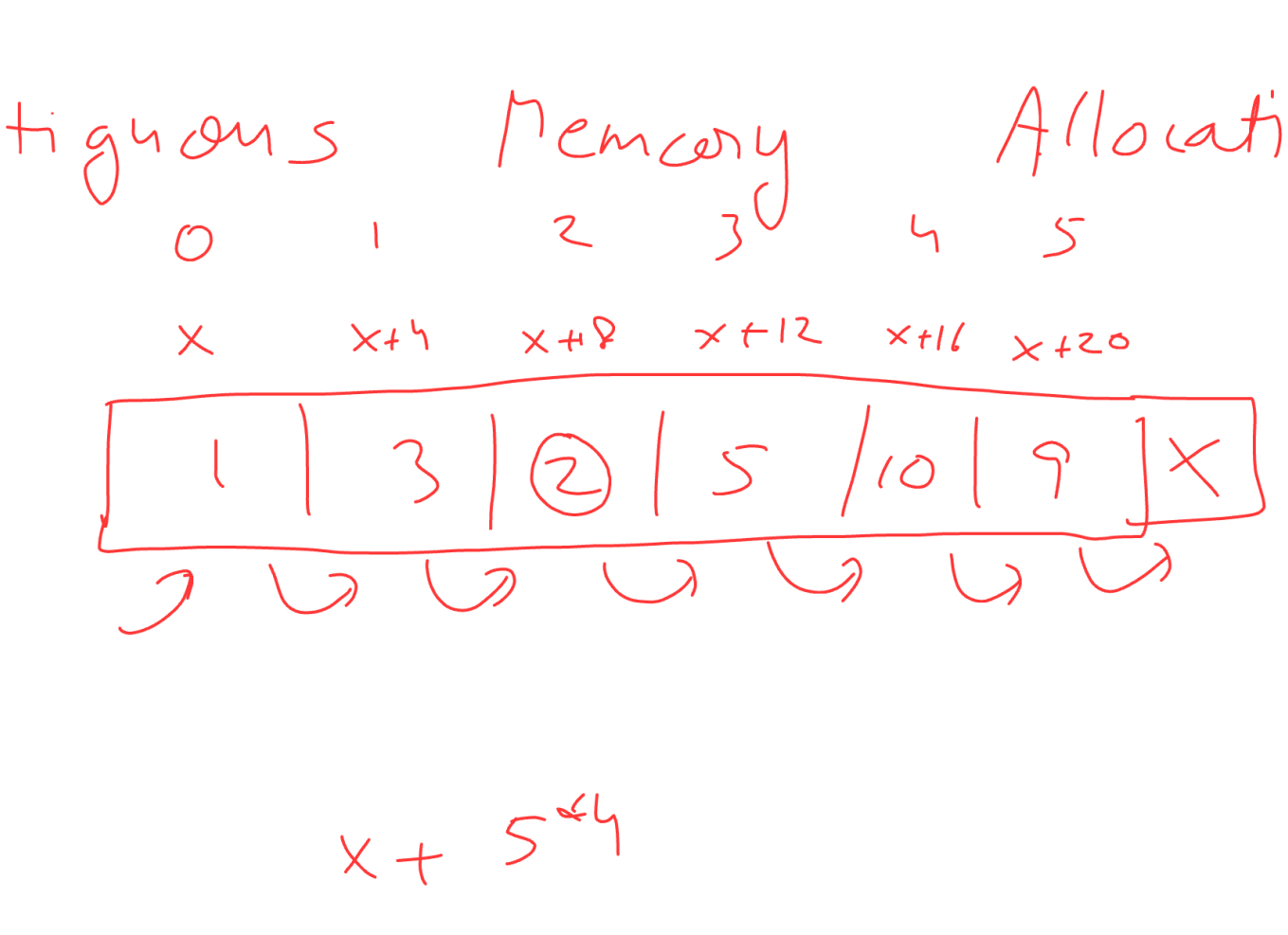
Del from the middle : O(n)

Adv.

1) Random Access
2) Cache Friendly

Dis adv.

1) Ins / Del from the beg / middle is costly
2)



1
2
4
8
16
32
64
128
256

[5, 4, 2, 3] o/p = 1

[0, -10, 1, 3, -20] o/p = 2

[1, 2, 3, 4, 5] o/p = 6

BF:

TC: $O(n^2)$

AS: $O(1)$

```
for (int i = 1 ; i <= n; i++) {
    Linear Search for i
    if found, move ahead.
    otherwise return i
}
return n+1;
```

Sorting: [-20, -10, 0, 1, 3]

[2, 3, 4, 5]

[-20, -10, 0, 1, 1, 1, 2, 3, 5].

val = ~~1 2 3~~ 4

TC: O(n logn + n) = O(n log n)

AS: O(1)

Hashing ~~Partitioning~~ Tech

	0	1	2	3	4	5	6	7	8
arr:	-20	-10	0	1	1	1	2	3	5

0	1	2	3	4	5	6	7	8	9
X	T	T	T		T				

o/p = 4.

TC: O(n)

AS: O(n)

Bucketing Tech.

0	1	2	3	4	5	6
-5	-6	-1	3	-2	-8	50

o/p = 4.

Use the i^{th} bucket to store the info whether (i+1) is present in the array or not.

TC: O(n)

AS: O(1)

$$\begin{array}{c|ccc} & 0 & 1 & 2 \\ \hline 0 & 1 & 0 & 1 \\ 1 & 1 & 1 & 1 \\ 2 & 1 & 0 & 1 \end{array} \longrightarrow \begin{array}{ccc} 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{array}$$

→ rows:

0	1	2
T	F	T

cols:

0	1	2
F	T	F

x0 0 x0
1 x0 1
1 0 1

1 1 1 1
1 ¦ 0 1 1
→ 1 ¦ 1 1 0
1 ¦ 1 1 1
1 ¦ 1 0 1

1 x0 x0 x0
0x ¦ 0 1 1
→ 0x ¦ 1 1 0 ⇒
1 ¦ 1 1 1
0x ¦ 1 0 1

1 x0 x0 x0
0x ¦ 0 x0 x0
→ 0x ¦ x0 x0 0
1 ¦ x0 x0 x0
0x ¦ x0 0 x0

↓

1 0 0 0
0 0 0 0
0 0 0 0
1 0 0 0
0 0 0 0

1 0 1
1 ¦ x0 1 ⇒
0x ¦ 0 x0

1 0 1
1 ¦ 0 1
0 ¦ 0 0

[zerothRowZero = True
zerothColZero = False.

1 0 1
1 ¦ x0 1 →
0x ¦ 0 x0

0x 0 x0
1 0 1
0 0 0

[5, 3, 2, 4, 6]

Subarrays.

[5, 3] ✓ [5, 3, 2] ✓ [5, 2, 4] ✗

[4] ✓ [ ] ✓ [3, 5] ✗ [5, 2, 3] ✗

Subsequence.

[5] ✓ [5, 3] ✓ [5, 2] ✓ [3, 4, 2] ✗

[1, 2, 3, -2, 5] o/p = 9 6/8/7/⑨

[-1, -2, -3, -4] o/p = -1